

Cloud Computing Programming Assignment 2

Harshita Gupta
hg365

Link to the github: <https://github.com/harshita-g/CloudComputing-ProgrammingAssignment2.git>

Link to the docker hub: <https://hub.docker.com/repository/docker/hg365/wine-predictor/general>

In this assignment we have to train the machine learning models on the given Wine dataset.

The task is done in parallel on multiple EC2 instances.

Step 1: We start the AWS lab and create 4 EC2 instances (Ubuntu). 1 instance will be the master node and other 3 will be the slave node, 1 node is used to perform the wine quality prediction.

Step 2: After creating the instances we connect to these instances using ssh command as shown below:

```
ssh -i C:\Users\harsh\Downloads\cloud-computing-p2.pem ubuntu@54.84.243.176
```

```
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '54.84.243.176' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.2 LTS (GNU/Linux 6.8.0-1027-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Wed Apr 30 17:49:43 UTC 2025

System load:  0.0               Processes:    115
Usage of /:   59.9% of 6.71GB   Users logged in: 0
Memory usage: 6%               IPv4 address for enX0: 172.31.93.42
Swap usage:   0%

 * Ubuntu Pro delivers the most comprehensive open source security and
   compliance features.

https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Tue Apr 29 14:33:41 2025 from 74.102.159.112
ubuntu@ip-172-31-93-42:~$ |
```

Step 3: Now in each of the instance we have to install spark.java and other required programs, this can be done using the following commands:

```
sudo apt update && sudo apt upgrade -y
sudo apt install openjdk-8-jdk -y
sudo apt install scala -y
wget https://archive.apache.org/dist/spark/spark-3.4.1/spark-3.4.1-bin-hadoop3.tgz
tar -xvzf spark-3.4.1-bin-hadoop3.tgz
sudo mv spark-3.4.1-bin-hadoop3 /opt/spark
tar -xvzf spark-3.4.1-bin-hadoop3.tgz
sudo mv spark-3.4.1-bin-hadoop3 /opt/spark
```

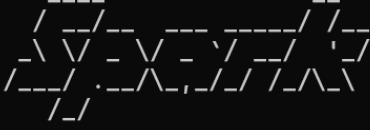
->Setting the environment variables

```
echo 'export SPARK_HOME=/opt/spark' >> ~/.bashrc
echo 'export PATH=$SPARK_HOME/bin:$PATH' >> ~/.bashrc
source ~/.bashrc
```

->verification of correct installation:

```
spark-shell --version
```

```
ubuntu@ip-172-31-93-42:~$ spark-shell --version  
Welcome to
```

```
        version 3.4.1  
  
Using Scala version 2.12.17, OpenJDK 64-Bit Server VM, 1.8.0_442  
Branch HEAD  
Compiled by user centos on 2023-06-19T23:01:01Z  
Revision 6b1ff22dde1ead51cbf370be6e48a802daae58b6  
Url https://github.com/apache/spark  
Type --help for more information.
```

Step 4: Now we need to add the credentials to each of the nodes using the access id, secret key and session token created by aws as follows:

```
vi ~/.aws/credentials
```

```
[default]
aws_access_key_id=ASIA4WFBGNJBWTSY3WB
aws_secret_access_key=vP9MZt3FRRFY0NLDrcgr2FQZIQFtERvz9Uusy+X8A
aws_session_token=IQoJb3JpZ2luX2VjEBIaCXVzLXdlc3QtMiJHMEUCIQCFsCYERzjEokpBF4ekIjdnO+Q3R8VRMDW8ZpVfkUmXSwIgvIy
m0UuAqsQIIq//////////ARAAGgw4NzIyMTY0MjI5NzgiDKfdkB+gE1KLp3E/CqFAojlMtgVfocL/PHA5ecPGb23GXTN+gp0/0+gYSc45Zy
/ROFzg+3xVAtiTG0FI3Qs3mjJE3HWR2LKH9iEjfv61KhPsIoepxUyZgpXqLhMugMJpWcbBXXp+akd1rbFEXR/muTtdtmT0R91xgU3QPbUYr
JMHmjS0rrVcXRakY6vEUGf5zbLgKobktxs2utfPIn08FoZUmRbFavU8EZ7T4SQ0x9GXfGZzkLPoSG3yRzRljL2qVByNWQKk0x+dWRFewoSqL
1ERInpoWD660trAOGLTx+b+Ct+ehmMa+LJfATQET+i5sQxEXBfAptVWeNLMQWh8vsn35tUmKSLFkKTuBAfMDfX6BZ8549dDJxamZK7GMSfBFf3
Im0tVb5upJY9ZlGnytZu/dIsi0mSx7RCs78kfQfxZaTcvkLFpcDalukJMoP5A=
~
~
~
~
~
~
~
~
~
~
~
```

Step 5: Now create a S3 bucket on AWS and upload the training dataset and validation dataset files.

☐	 TrainingDataset.csv	csv	April 28, 2025, 23:15:22 (UTC-04:00)	67.2 KB	Standard
☐	 ValidationDataset.csv	csv	April 29, 2025, 10:41:37 (UTC-04:00)	8.6 KB	Standard

We can validate if we can access the bucket on our nodes or not as follows:

aws s3 ls s3://wine-train-dataset-pa2

```
2025-04-29 03:15:22      68804 TrainingDataset.csv
2025-04-29 14:41:37      8760 ValidationDataset.csv
ubuntu@ip-172-31-88-42:~$ |
```

Step 6: Create a maven project using:

```
sudo apt install maven -y
mvn archetype:generate \
  -DgroupId=com.winequality \
  -DartifactId=WineQualityTrainer \
  -DarchetypeArtifactId=maven-archetype-quickstart \
  -DinteractiveMode=false
```

Now we have a project folder called WineQualityTrainer:

```
ubuntu@ip-172-31-88-42:~$ ls
WineQualityTrainer snap spark-3.4.1-bin-hadoop3.tgz
ubuntu@ip-172-31-88-42:~$ |
```

Then we go inside the WineQualityTrainer folder and edit the pom.xml file to add the dependencies of Spark and MLlib.

Now pom.xml file will look like this:

```
        </dependency>
        <dependency>
            <groupId>org.apache.spark</groupId>
            <artifactId>spark-core_2.12</artifactId>
            <version>3.4.1</version>
        </dependency>
    </dependencies>
    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.1</version>
                <configuration>
                    <source>1.8</source>
                    <target>1.8</target>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
"pom.xml" 47L, 1471B
```

Step 7: Now we need to add the code file: App.java which contains the code to train the models on training dataset.

```

import org.apache.spark.sql.Dataset;
import org.apache.spark.sql.Row;
import org.apache.spark.sql.SparkSession;
import org.apache.spark.sql.types.DataTypes;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import static org.apache.spark.sql.functions.col;
public class App {
    // data preprocessing
    public static Dataset<Row> cleanData(Dataset<Row> df) {
        for (String colName : df.columns()) {
            String cleanColName = colName.replace("\\", "");
            df = df.withColumn(cleanColName, col(colName).cast(DataTypes.DoubleType));
            if (!cleanColName.equals(colName)) {
                df = df.drop(colName);
            }
        }
        return df;
    }
    public static void main(String[] args) {

        SparkSession spark = SparkSession.builder()
            .appName("wine_quality_prediction")
            .master("local[*]")
            .getOrCreate();

        JavaSparkContext sc = new JavaSparkContext(spark.sparkContext());
        sc.setLogLevel("ERROR");
    }
}

```

We have used the following metrics: accuracy, F1 score, precision and recall.

Step 8: Build the above code file using: mvn clean package and on successful build we see the below result:

```

Results :

Tests run: 1, Failures: 0, Errors: 0, Skipped: 0

[INFO]
[INFO] --- maven-jar-plugin:2.4:jar (default-jar) @ WineQualityTrainer ---
[INFO] Building jar: /home/ubuntu/WineQualityTrainer/target/WineQualityTrainer-1.0-SNAPSHOT.jar
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 3.477 s
[INFO] Finished at: 2025-04-30T18:36:27Z
[INFO]

```

Step 9: Now we will start the master and worker nodes using:

To start the master node: /opt/spark/sbin/start-master.sh

Can confirm using 'jps' command that the Master node is up and running

```

ubuntu@ip-172-31-88-42:~/WineQualityTrainer$ /opt/spark/sbin/start-master.sh
starting org.apache.spark.deploy.master.Master, logging to /opt/spark/logs/spark-ubuntu
2-31-88-42.out
ubuntu@ip-172-31-88-42:~/WineQualityTrainer$ jps
1834 Master
1882 Jps

```

To start the worker nodes: /opt/spark/sbin/start-worker.sh spark://172.31.88.42:7077

```
ubuntu@ip-172-31-93-42:~$ /opt/spark/sbin/start-worker.sh spark://172.31.88.42:7077
starting org.apache.spark.deploy.worker.Worker, logging to /opt/spark/logs/spark-ubuntu-01
2-31-93-42.out
ubuntu@ip-172-31-93-42:~$ jps
2164 Worker
2215 Jps
```

Now we need to submit the job and the master will distribute the work to its worker nodes. We are using the below command, here the task will be distributed to all the workers connected to the master node and the training is done in parallel on EC2 instances.

```
/opt/spark/bin/spark-submit \
--class com.winequality.App \
--master spark://172.31.88.42:7077 \
--deploy-mode client \
--executor-memory 1G \
--jars /opt/spark/jars/hadoop-aws-3.3.4.jar,/opt/spark/jars/aws-java-sdk-bundle-1.12.262.jar \
--conf spark.hadoop.fs.s3a.access.key=ASIA4WFBGNJBWTSY3WB \
--conf spark.hadoop.fs.s3a.secret.key=vP9MZt3FRRfY0NIDrcgr2FQZIQFtERvz9Usy+X8A \
--conf
spark.hadoop.fs.s3a.session.token=IQoJb3JpZ2luX2VjEBIaCXVzLXdlc3QtMiJHMEUCIQCFsC
YERzjEokpBF4ekIjdnO+Q3R8VRMDW8ZpVfkUmXSwlgViye54dbmYv2jXsgcLzVLtyx5dq5zpWjr
Ah9jgm0UhAqsQlIq////////ARAAGgw4NzlyMTY0MjI5NzgiDKfdkB+gE1KLIP3E/CqFAojlmTgVfo
cl/PHA5ecPGb23GXTN+gp0/0+gYSc45ZysK5mslZlhal54ktagX6rJCs/uKpMbCrVlwa/ROFzg+3x
VAtiTg0FI3Qs3mjJYE3HWR2LKH9iEjfvR61KhPslloepxUyZgpXqLhMugMJpWcbBXXp+akd1rbFE
xR/muTtdtmT0R91xgU3QPbUYrVOQLvaBVgtpcbUnSwvNCO6z7U8J3tBrKBfBJMHmjSOrrVcX
RakY6vEUGf5zbLgKobktxs2utfPlnO8FoZUmRbFavU8EZ7T4SQ0x9GXfGZzkIPoSG3yRzRlJL2q
VByNWQKk0x+dWRFewoSuQl4BbfUHYdSilk5YRA2DjCQwcnABjqdASjK+k1ERInpoWD660trA
OGITxb+Ct+ehmMa+LJfATQET+i5sQxEXBfAptVWeNLmQWh8vsN35tUmKSIFkKTuBAfMDfX6B
Z8549dDJxamZK7GMSfBFf3vQseL1omkd5eAXBIMKxRpa7MRrR59MF52Vflm0tVb5upJY9ZIGn
ytZu/dlSiOmSx7RCs78KfQfxZaTcvKIFpcDalukJMoP5A= \
--conf spark.hadoop.fs.s3a.impl=org.apache.hadoop.fs.s3a.S3AFileSystem \
--conf spark.hadoop.fs.s3a.endpoint=s3.amazonaws.com \
target/WineQualityTrainer-1.0-SNAPSHOT.jar
```

```
Training and evaluating: LogisticRegression
Model: LogisticRegression | Accuracy: 0.4125 | F1 Score: 0.2409 | Precision: 0.1702 | Recall: 0.4125
Model saved to: s3a://wine-train-dataset-pa2/LogisticRegression
Training and evaluating: RandomForestClassifier
Model: RandomForestClassifier | Accuracy: 0.5500 | F1 Score: 0.5282 | Precision: 0.5127 | Recall: 0.5500
Model saved to: s3a://wine-train-dataset-pa2/RandomForestClassifier
Training and evaluating: DecisionTreeClassifier
Model: DecisionTreeClassifier | Accuracy: 0.4813 | F1 Score: 0.4621 | Precision: 0.4586 | Recall: 0.4813
Model saved to: s3a://wine-train-dataset-pa2/DecisionTreeClassifier
Training and evaluating: NaiveBayes
Model: NaiveBayes | Accuracy: 0.4563 | F1 Score: 0.4573 | Precision: 0.4944 | Recall: 0.4563
Model saved to: s3a://wine-train-dataset-pa2/NaiveBayes
Training and evaluating: MultilayerPerceptronClassifier
Model: MultilayerPerceptronClassifier | Accuracy: 0.5750 | F1 Score: 0.5581 | Precision: 0.5572 | Recall: 0.5750
Model saved to: s3a://wine-train-dataset-pa2/MultilayerPerceptronClassifier
```

Step 10: Now we can see that the models are saved in the S3 buckets.

Objects (7)

[Copy S3 URI](#)[Copy URL](#)[Download](#)[Open](#)[Delete](#)[Actions](#)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you need to set permissions. [Learn more](#)

☐	Name	Type	Last modified	Size	Storage class
☐	DecisionTreeClassifier/	Folder	-	-	-
☐	LogisticRegression/	Folder	-	-	-
☐	MultilayerPerceptronClassifier/	Folder	-	-	-
☐	NaiveBayes/	Folder	-	-	-
☐	RandomForestClassifier/	Folder	-	-	-

Step 11: Now we will add the code for predictor function: AppPredict.java and build the file using mvn clean package and the following output can be seen:

```
Results :

Tests run: 1, Failures: 0, Errors: 0, Skipped: 0

[INFO] --- maven-jar-plugin:2.4:jar (default-jar) @ WineQualityTrainer ---
[INFO] Building jar: /home/ubuntu/WineQualityTrainer/target/WineQualityTrainer-1.0-SNAPSHOT.jar
[INFO] BUILD SUCCESS
[INFO] Total time: 3.202 s
[INFO] Finished at: 2025-04-30T19:40:45Z
[INFO]
```

Step 12: Now submit the job to spark using this command:

```
/opt/spark/bin/spark-submit \
--class com.winequality.AppPredict \
--master spark://172.31.88.42:7077 \
--deploy-mode client \
--executor-memory 1G \
--jars /opt/spark/jars/hadoop-aws-3.3.4.jar,/opt/spark/jars/aws-java-sdk-bundle-1.12.262.jar \
--conf spark.hadoop.fs.s3a.access.key=ASIA4WFBGNJBWTSY3WB \
--conf spark.hadoop.fs.s3a.secret.key=vP9MZt3FRRfY0NIDrcgr2FQZIQFtERvz9Uusy+X8A \
--conf spark.hadoop.fs.s3a.session.token=IQoJb3JpZ2luX2VjEBIaCXVzLXdlc3QtMiJHMEUCIQCFsC
YERZjEokpBF4ekljdnO+Q3R8VRMDW8ZpVfkUmXSwlgViye54dbmYv2jXsgcLzVLtyx5dq5zpWjr
Ah9jgm0UhAqsQIlq////////ARAAGgw4NzlyMTY0Mjl5NzgiDKfdkB+gE1KLIP3E/CqFAojlmTgVfo
cl/PHA5ecPGb23GXTN+gp0/0+gYSc45ZysK5mslZlhal54ktagX6rJCs/uKpMbCrVlwa/ROFzg+3x
VAtiTg0FI3Qs3mjJYE3HWR2LKH9iEjfVr61KhPsloepxUyZgpXqLhMugMJpWcbBXXp+akd1rbFE
xR/muTdtmT0R91xgU3QPbUYrVOQLvaBVgtpcbUnSwvNCO6z7U8J3tBrKBfBJMHmjSOrrVcX
RakY6vEUGf5zbLgKobktxs2utfPlnO8FoZUmRbFavU8EZ7T4SQ0x9GXfGZzkIPoSG3yRzRIjL2q
```

```
VByNWQKk0x+dWRFewoSuQl4BbfUHYdSilk5YRA2DjCQwcnABjqdASjK+k1ERInpoWD660trA
OGITxb+Ct+ehmMa+LJfATQET+i5sQxEXBfAptVWeNLmQWh8vs35tUmKSIFkKTuBAfMDfX6B
Z8549dDJxamZK7GMSfBFf3vQseL1omkd5eAXBIMKxRpa7MRrR59MF52Vflm0tVb5upJY9ZIGn
ytZu/dlSiOmSx7RCs78KfQfxZaTcvKIFpcDalukJMoP5A= \
--conf spark.hadoop.fs.s3a.impl=org.apache.hadoop.fs.s3a.S3AFileSystem \
--conf spark.hadoop.fs.s3a.endpoint=s3.amazonaws.com \
target/WineQualityTrainer-1.0-SNAPSHOT.jar
```

```
Evaluating model: LogisticRegression
Model: LogisticRegression | Accuracy: 0.4125 | F1 Score: 0.2409 | Precision: 0.1702 | Recall: 0.4125

Evaluating model: RandomForestClassifier
Model: RandomForestClassifier | Accuracy: 0.5500 | F1 Score: 0.5282 | Precision: 0.5127 | Recall: 0.5500

Evaluating model: DecisionTreeClassifier
Model: DecisionTreeClassifier | Accuracy: 0.4813 | F1 Score: 0.4621 | Precision: 0.4586 | Recall: 0.4813

Evaluating model: NaiveBayes
Model: NaiveBayes | Accuracy: 0.4563 | F1 Score: 0.4573 | Precision: 0.4944 | Recall: 0.4563

Evaluating model: MultilayerPerceptronClassifier
Model: MultilayerPerceptronClassifier | Accuracy: 0.5750 | F1 Score: 0.5581 | Precision: 0.5572 | Recall: 0.5750
```

Step 13: Creating the docker file and install the necessary dependencies,
 sudo snap install docker

Then create a 'Dockerfile' in the working directory and then run: docker build -t wine-predictor .

```
ubuntu@ip-172-31-88-42:~/WineQualityTrainer$ docker images
REPOSITORY          TAG          IMAGE ID          CREATED          SIZE
wine-predictor      latest      beba70c42c53     3 minutes ago   1.19GB
```

Step 14: Create account on docker hub and push the image:

docker tag wine-predictor hg365/wine-predictor

docker push hg365/wine-predictor

```
ubuntu@ip-172-31-88-42:~/WineQualityTrainer$ docker push hg365/wine-predictor
Using default tag: latest
The push refers to repository [docker.io/hg365/wine-predictor]
11c467c34908: Pushed
de9a3ab699cc: Pushed
387fc35cf0b5: Pushed
6b5aaff44254: Mounted from library/openjdk
53a0b163e995: Mounted from library/openjdk
b626401ef603: Mounted from library/openjdk
9b55156abf26: Mounted from library/openjdk
293d5db30c9f: Mounted from library/openjdk
03127cdb479b: Mounted from library/openjdk
9c742cd6c7a5: Mounted from library/openjdk
latest: digest: sha256:741ae376b58fc8ead6f02dc7f5f68afb26c1f17d90bf9782d0ce93f79466974a size: 2428
```

Link of the image: <https://hub.docker.com/repository/docker/hg365/wine-predictor/general>

Result:

For extra credit, trained 5 models and their performance metrics are reported as follows:

Model	Accuracy	F1 Score	Precision	Recall
Logistic Regression	0.4125	0.2409	0.1702	0.4125
Random Forest Classifier	0.5500	0.5282	0.5127	0.5500
Decision Tree Classifier	0.4813	0.4621	0.4586	0.4813
Naive Bayes	0.4563	0.4573	0.4944	0.4563
Multilayer Perceptron Classifier	0.5750	0.5581	0.5572	0.5750